

Taylor & Francis
Taylor & Francis Group

Journal of Information and Telecommunication

ISSN: 2475-1839 (Print) 2475-1847 (Online) Journal homepage: www.tandfonline.com/journals/tjit20

An efficient method for mining frequent weighted subgraphs based on weighted edges

Lam B. Q. Nguyen, Nhan H. Vo, Nhung N. Chau & Bay Vo

To cite this article: Lam B. Q. Nguyen, Nhan H. Vo, Nhung N. Chau & Bay Vo (2025) An efficient method for mining frequent weighted subgraphs based on weighted edges, Journal of Information and Telecommunication, 9:4, 543-558, DOI: 10.1080/24751839.2025.2500132

To link to this article: https://doi.org/10.1080/24751839.2025.2500132

© 2025 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group

Published online: 06 May 2025.

Submit your article to this journal

Article views: 683

View related articles

View Crossmark data

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=tjit20

JOURNAL OF INFORMATION AND TELECOMMUNICATION
2025, VOL. 9, NO. 4, 543–558
https://doi.org/10.1080/24751839.2025.2500132

OPEN ACCESS Check for updates

An efficient method for mining frequent weighted subgraphs based on weighted edges

Lam B. Q. Nguyen[a], Nhan H. Vo[a], Nhung N. Chau[a] and Bay Vo [b]

[a]Faculty of Information and Communications, Kiengiang University, Kiengiang, Vietnam; [b]Faculty of Information Technology, HUTECH University, Ho Chi Minh city, Vietnam

ABSTRACT

Since the GraMi algorithm was introduced to mine frequent subgraphs, many other algorithms based it have also been published. The OWGraMi (Optimized Weighted Graph Mining) published in 2022 can be considered a state-of-the-art algorithm for mining frequent subgraphs in a single large weighted graph. However, the OWGraMi algorithm is still limited when it only considers the weights of vertices on the graph, not edge weights. In this paper, we introduce two new contributions to further improve the OWGraMi algorithm: calculating weights for subgraphs based on their edge weights, and using these edge weights to prune the search space to increase the performance of the new weighted graph mining algorithm, named WEGM (Weighted Edge GraMi). Our new algorithm demonstrates that it outperforms OWGraMi in many evaluation criteria, such as search space, processing time and memory consumption.

ARTICLE HISTORY
Received 13 March 2025
Accepted 26 April 2025

KEYWORDS
Data mining; graph mining; weighted subgraph; pruning techniques; frequent subgraph mining

1. Introduction

Large weighted graphs with weights on both vertices and edges are increasingly widely used for simulating, analysing, and storing complex systems (Elseidy et al., 2014; Nguyen, 2022; Svitáková et al., 2021) such as Data Mining, Recommender Systems, Social Networks, Computer Network Models, Traffic Flow, and Crime Investigation, etc. Moreover, mining large weighted graphs has also attracted the attention of many researchers (Elseidy et al., 2014; Franciscus et al., 2019; Nguyen, 2022; Nguyen et al., 2022ba; Zhong, 2024). For example, Figure 1 is a graph simulating a network of researchers, in which each vertex *u*, labelled P (Professor), A (Associate Professor), D (Doctor) and M (Master) represents a researcher (with a red reputation index being the vertex weight), these researchers have mutual articles (the weight of each edge *e* indicates the number of articles a pair of researchers have).

The problem is that we need to find research groups in which the members must have a large enough number of mutual articles (*Max*), or find groups of products (in a company) with few buyers during the year (*Min*) so that the related company can adjust its sales campaign

CONTACT Bay Vo vd.bay@hutech.edu.vn Faculty of Information Technology, HUTECH University, Ho Chi Minh city 700000, Vietnam

© 2025 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group
This is an Open Access article distributed under the terms of the Creative Commons Attribution License (http://creativecommons.org/licenses/by/4.0/), which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited. The terms on which this article has been published allow the posting of the Accepted Manuscript in a repository by the author(s) or with their consent.

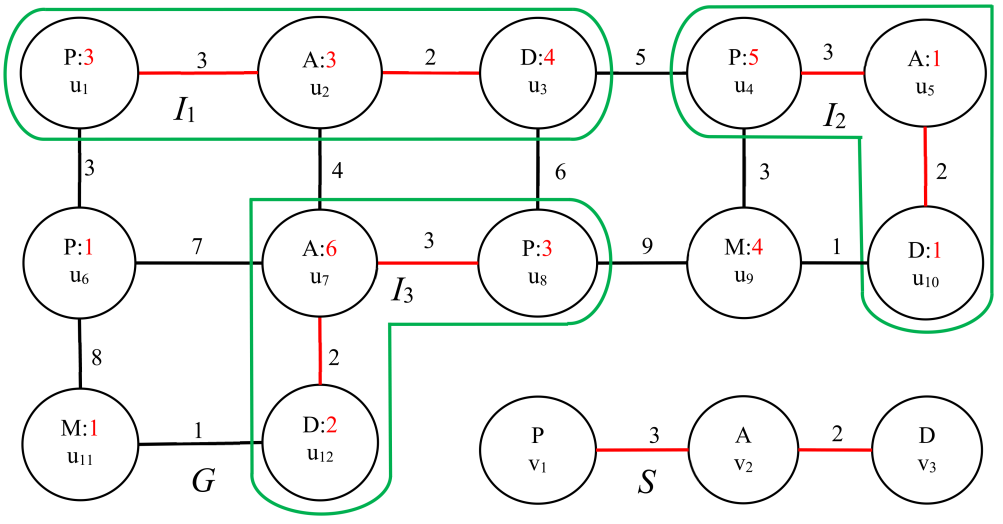


Figure 1. Large graph *G* and subgraph *S*.

(Bui et al., 2018; Chen et al., 2024; Fournier-Viger et al., 2019; Nguyen et al., 2019) in a number of recommender systems, social network analysis and mining systems, and so on.

In 2020, the WeGraMi algorithm (Le et al., 2020) was proposed to mine weighted subgraphs in a large single graph with weights on vertices. In 2022, an improved version of this algorithm was proposed to prune the search space using weight thresholds, called the Optimized Weighted Graph Mining (OWGraMi) algorithm (Le et al., 2022). Although it has superior performance compared to the original algorithm, OWGraMi still only calculates the weights on the vertices of the subgraph, not the weights on the edges, which is the motivation for us to improve our algorithm in this paper.

Our new algorithm (WEGM) has the following two main contributions:

(1) Add a function of weighting on the edges to OWGraMi algorithm (Le et al., 2022): the original algorithm only calculates weights for subgraphs based on vertices, the new algorithm can weight both the vertices and edges of subgraphs. In this paper, we propose two weighting methods, *Max* and *Min*, and users can choose how to calculate edge weights based on their research purposes.
(2) Prune the search space based on edge weights: our new algorithm sets weight thresholds and removes frequent edges that do not satisfy the given threshold, helping the algorithm significantly reduce the search space, thereby increasing mining performance in terms of time and storage space.

Based on (2), early in the process we prune a lot of candidate subgraphs that do not satisfy the threshold (details in section IV). Therefore, our new algorithm has superior mining performance compared to the original algorithm on all three evaluation criteria: search space, execution time and memory usage (details in section V).

Our paper is organized in five parts, as follows: Section II introduces graph mining algorithms related to the new algorithm; Section III consist of definitions of graph

mining; Section IV describes in detail our two contributions to the WEGM algorithm; Section V presents the results of the experiment and compares the WEGM algorithm with the original algorithm on three evaluation criteria; Section VI gives our conclusion and some directions for future development.

2. Related works

In 2014, the GraMi algorithm (Elseidy et al., 2014) was proposed as a fast algorithm to mine frequent subgraphs from a single large graph. GraMi stores templates of candidate subgraphs, searches for isomorphisms and marks the values (vertices) in the domains of candidate subgraphs, but does not store all appearances of each subgraph in an entire large graph (Sahu et al., 2023). In 2020, the GraMi algorithm was improved to be able to find the support for frequent subgraphs, with the new approach called SuGraMi (Support GraMi) (Nguyen et al., 2020). In the same year, FlexGraph (Park et al., 2020) was a scalable distributed graph mining algorithm (Svitáková et al., 2021) that reduced the cost of exploiting the properties of real-world graphs. Specifically, it decreased the communication cost, the bottleneck of distributed systems. Furthermore, this method had a flexible storage format, reducing I/O costs.

Although SuGraMi can find all frequent subgraphs with their full support in a large single graph, this algorithm still cannot find closed subgraphs. Therefore, in 2021, CloGraMi (Closed GraMi) (Nguyen et al., 2021a) was proposed as an algorithm to mine closed subgraphs and included two effective strategies to optimize this approach. In order to reduce the search time a new tree traversal strategy was added to CloGraMi, as well as a condition to determine if a subgraph is closed subgraph or not, and remove it from the search tree during mining process if needed. Also in 2021, a new algorithm was proposed based on connected components with a new isomorphism searching method, Connected Components GraMi (CCGraMi) (Nguyen et al., 2021b).

In 2022, a new method to balance the search space (Talukder & Zaki, 2016) during the mining process was introduced, named Balancing GraMi, or BaGraMi (Nguyen et al., 2022ba). In the same year an enhanced closed subgraph mining algorithm with a new parallel processing solution, called Parallel Closed GraMi (PCGraMi) (Nguyen et al., 2022) was introduced. In 2023, So-GPARs was proposed for mining association rules from common subgraphs (Huynh et al., 2024). However, many problems still remain with frequent subgraph mining (Liu et al., 2020), and these have also been summarized in the literature (Nguyen et al., 2022bb). The DIMMining algorithm (Dai et al., 2022) was proposed with four techniques for more efficient pruning and the use of a parallel strategy.

For the weighted subgraph mining problem (Preti et al., 2021), in 2020, the WeGraMi algorithm (Weighted GraMi) (Le et al., 2020) was proposed using the *MaxMin* strategy to calculate weights for each frequent subgraph in large graphs. In addition, this algorithm had another contribution to WeGraMi which is to add a parameter w (weight threshold) to improve performance when generating candidate subgraphs. This is an effective strategy to prune all subgraphs that do not satisfy the given weight threshold w early in the process.

In 2023, the AWeGraMi algorithm (Le et al., 2023) was proposed using a different metric to OWGraMi (Le et al., 2022). This algorithm uses the average value of all vertices in the

domain to calculate weights for the graph (Guzmán-Ponce et al., 2020). This algorithm also includes a method to effectively prune the search space of the mining process using the previous method *MaxMin* (Le et al., 2020), because the average measure does not ensure the Downward Closure Property (DCP) (Chen et al., 2024; Preti et al., 2023). However, AWeGraMi still calculates weights based on the vertices in the graph and does not consider the weight of the edges. Therefore, a new algorithm with an edge weight measure is needed for the weighted graph mining problem.

Also in 2023, SuMExplorer (Wangmo & Wiese, 2023) was introduced with visual exploratory subgraph search to support visual analysis (Li et al., 2022; Nguyen et al., 2022bb) and to ease user interaction. SuMExplorer algorithm addresses the problem to reduce the computational complexity of visual exploratory subgraph searching during the indexing stage. A compact summarization of input graph to reduce construction time and the index size; besides weighted frequent subgraph mining (Le et al., 2020) for novel graph summarization-based the visual subgraph searching; proposing a novel index structure representing the subgraphs extracted in a compact form. The proposed framework can provide a higher efficiency than the baseline on three real-world biological datasets.

In 2024, the So-GPARs association rule mining method (Fan et al., 2015; Huynh et al., 2024; Wang et al., 2020) was proposed to effectively reduce the computational cost and execution time, and the association rules can be determined in one phase immediately after mining frequent subgraphs without having to bear the cost of finding a set of parent frequent subgraphs to generate rules, unlike two-phase rule mining methods. The algorithm generates a set of association rules (Fan et al., 2015; Huynh et al., 2024) (representing the relationship between vertices in a graph) from a set of identified frequent subgraphs, mining association rules (Fan et al., 2015; Sasaki & Karras, 2024) to solve the problems of analysis of correlations and interactions related to vertex labels with data simulated by graphs (He et al., 2024).

Also in 2024, HUSM algorithm (Chen et al., 2024) was proposed, it defined a utility function in single graph databases, formally define the HUSM problem (high utility subgraph mining). Because of several challenges such as lack of DCP and a huge number of subgraph candidates (Fournier-Viger et al., 2019), HUSM algorithm designs several utility upper bounds satisfying the DCP and then develop the HUSM algorithm to mine all high utility subgraphs from single graph databases. For high utility subgraphs, this algorithm designs a lossless concise representation with fewer instances, makes the mining result more representative. In that paper, the experimental results demonstrate an excellent performance, the concise representation plays an important role in simplifying the mining high utility subgraphs results.

In 2025, a summary graph-induced Invariant Learning (SIL) (Ning et al., 2025) with experimental results that validate the effectiveness of this model is a promising strategy, it achieves generalizable graph learning tasks (Ren et al., 2025). Graph invariant learning identifies invariant subgraphs by selecting the important edges/nodes for the stable predictivity. The existing approaches are relying on subgraph predictions facing a challenge because the learned invariant subgraph contains spurious nodes, shows poor connectivity, reducing generalization power of Graph Neural Networks (GNNs) (Corso et al., 2024). This SIL model innovatively adopts a summary graph for learning strong connected, accurate invariant subgraphs to leverage the subgraph connectivity and

predictivity. Additionally, the SIL model not only provide a theoretical analysis for summary graph learning mechanism, but also guarantee that the learned summary graph is consistent with the original graph.

In the OWGraMi algorithm (Le et al., 2022), two new strategies were further proposed to reduce the number of candidates and the running time of calculating subgraph weights. In the WeGraMi algorithm (Le et al., 2020), (Le et al., 2022), candidate subgraphs are generated from a list of frequent edges in *fEdges* list this list contains all edges satisfying the frequency threshold *f* (Nguyen et al., 2020), if the frequent subgraph reaches the weight threshold *w* then it is a weighted subgraph (Le et al., 2020). This algorithm considers each edge in the list *fEdges* as a 1-edge subgraph and removes all of them that cannot reach the weight threshold *w*, and thus significantly reduces the number of candidates generated. In algorithm (Le et al., 2020) a *MaxMin* method was introduced to calculate the weights for all frequent subgraphs. However, when a subgraph is determined as a frequent subgraph, the process calculates the weight by traversing all the values in its domain, which takes a lot of time. Since OWGraMi introduces a new method to quickly calculate the weights for each frequent subgraph based on the weight of its parent.

3. Definitions and propositions

Definition 1 (Le et al., 2020, 2022): A large weighted graph $G = (V, E, L, W)$, containing vertex set V, edge set E, function L to assign labels to vertices $v \in V$, and function W to assign weights for all vertices $v \in V$, all edges $e \in E$.

Definition 2 (Le et al., 2020, 2022): A subgraph $S = (V_S, E_S, L_S, W_S)$ of G, where $V_S \subseteq V$, $E_S \subseteq E$, $L_S \subseteq L$, $W_S(u) = W(v)$ for all vertices $u \in V_S$, $v \in V$ and $W_S(d) = W(e)$ for all edges $d \in E_S$, $e \in E$.

For example, in Figure 1, G is a large weighted graph and S is a subgraph of G.

Definition 3 (Elseidy et al., 2014; Nguyen, 2022; Nguyen et al., 2020): An isomorphism I of S in G is a function f: $V_S \rightarrow V$ is:

(a) $L_S(v) = L(f(v))$ for all vertices $v \in V_S$, and
(b) $(f(u), f(v)) \in E$ and $W_S(u, v) = W(f(u), f(v))$ for all edges $(u, v) \in E_S$.

S	$\{v_1, v_2, v_3\}$	(v_1, v_2)	(v_2, v_3)
I_1	$\{u_1, u_2, u_3\}$	(u_1, u_2)	(u_2, u_3)
I_2	$\{u_4, u_5, u_{10}\}$	(u_4, u_5)	(u_5, u_{10})
I_3	$\{u_8, u_7, u_{12}\}$	(u_8, u_7)	(u_7, u_{12})

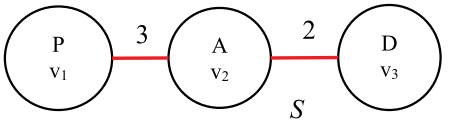


Figure 2. Isomorphisms of S in G.

For example, in Figure 1, subgraph S has three isomorphisms I_1, I_2 and I_3 in G as shown in Figure 2.

Proposition 1. *An assignment in the domain of a vertex u (in G) for variable v (in S) is valid if there exists a corresponding isomorphism I that assigns u to v.*

For example, in Figure 2 there does not exist any isomorphism I assigning u_6 to v_1, therefore u_6 in Figure 3 is an invalid assignment.

Definition 4 (Nguyen et al., 2020, 2022): A subgraph S is determined as frequent in a large graph G if its support (MNI) (Nguyen et al., 2022ba), denoted by $s_G(S)$, is greater than or equal to a given frequency threshold f.

For example, in Figure 1, S has three distinct isomorphisms as in Figure 2:

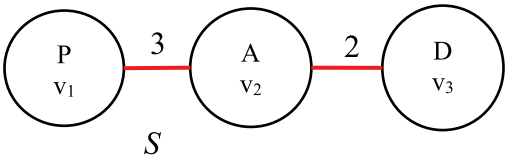
$$s_G(S) = min\{|D(v_1)|, |D(v_2)|, |D(v_3)|\}$$
$$= min\{|(u_1, u_4, u_8)|, |(u_2, u_5, u_7)|, |(u_3, u_{10}, u_{12})|\} = min\{3, 3, 3\} = 3$$

Definition 5 (Le et al., 2020, 2022): The vertex weight of subgraph S in G, denoted $W_V(S)$, is the weight calculated by the *MaxMin* method (detail in the OWGraMi algorithm Le et al., 2022).

For example:

$$W(v_1) = Max(W(u_1), W(u_4), W(u_8)) = Max(3, 5, 3) = 5$$
$$W(v_2) = Max(W(u_2), W(u_5), W(u_7)) = Max(3, 1, 6) = 6$$
$$W(v_3) = Max(W(u_3), W(u_{10}), W(u_{12})) = Max(4, 1, 2) = 5$$
$$W_V(S) = Min(W(v_1), W(v_1), W(v_1)) = Min(5, 6, 5) = 5$$

Because in the OWGraMi algorithm we proposed a vertex-based weighting method, in this paper we only focus on edge-based weighting method.



Variables and domains		
v1: 5	v2: 6	v3:4
u1:3	u2:3	u3:4
u4:5	u5:1	u10:1
~~u6:1~~	u7:6	u12:2
u8:3		

~~ *Invalid assignments*

Figure 3. Subgraph S and its domain.

Definition 6: The edge weight of subgraph S in G, denoted $W_E(S)$, is the maximum (or minimum) value of all the weights on edges e in S.

$$W_E min(S) = Min(W(v_1, v_2), W(v_2, v_3)) = Min(3, 2) = 2.$$
$$W_E max(S) = Max(W(v_1, v_2), W(v_2, v_3)) = Max(3, 2) = 3.$$

Definition 7 (Le et al., 2020, 2022): A weighted subgraph S in G is a frequent subgraph that has vertex weights $W_V(S)$ and edge weights $W_E(S)$ satisfying a weight threshold w (given by the user).

Definition 8 (Le et al., 2020, 2022): Unweighted subgraph S is an infrequent subgraph or a frequent subgraph whose vertex weight or edge weight do not satisfy the weight threshold w (given by the user).

In the WeGraMi and OWGraMi algorithms, we proposed the *MaxMin* method using *vertex weights* to calculate the weight for all frequent subgraphs in a large weighted graph. However, this *MaxMin* method only calculates weights based on the vertices of the subgraph, but does not consider edges. In this paper, we add a new weighting method w to the OWGraMi algorithm based on the *edge weights* contained in the subgraph. As the first contribution, we use the *Max* and *Min* operators to calculate the weights for frequent subgraphs. As the second contribution, we use Definition 8 to early prune subgraphs that do not reach the weight threshold w, increasing the performance of the OWGraMi algorithm, and we call it WEGM (Weighted Edge GraMi).

4. Proposed algorithm

In this paper, we propose the WEGM algorithm based on the OWGraMi algorithm, with the improvement of calculating weights on both vertices and edges. We provide three weighting methods for a frequent subgraph that users choose for their different research purposes.

- *MaxMin* method: Calculate weights based on vertices like the original OWGraMi algorithm.
- *Max* method: The weight of a frequent subgraph S is the maximum weight of the edges in S.

$$W_{max}(S) = Max(W(e)), \forall e \in E_S$$

- *Min* method: The weight of a frequent subgraph S is the minimum weight of the edges in S.

$$W_{min}(S) = Min(W(e)), \forall e \in E_S$$

Because the *MaxMin* method (Le et al., 2022) has been presented in detail in the OWGraMi algorithm, in this paper we will focus on describing the two methods *Max* and *Min* based on edge weights. Besides, we will prune the search space to improve the performance of WEGM based on these two methods.

First, WEGM will read all edges in the large graph G (from dataset) to find a list of frequent edges *fEdges* (the complexity similar to WeGraMi and OWGraMi). For example, in

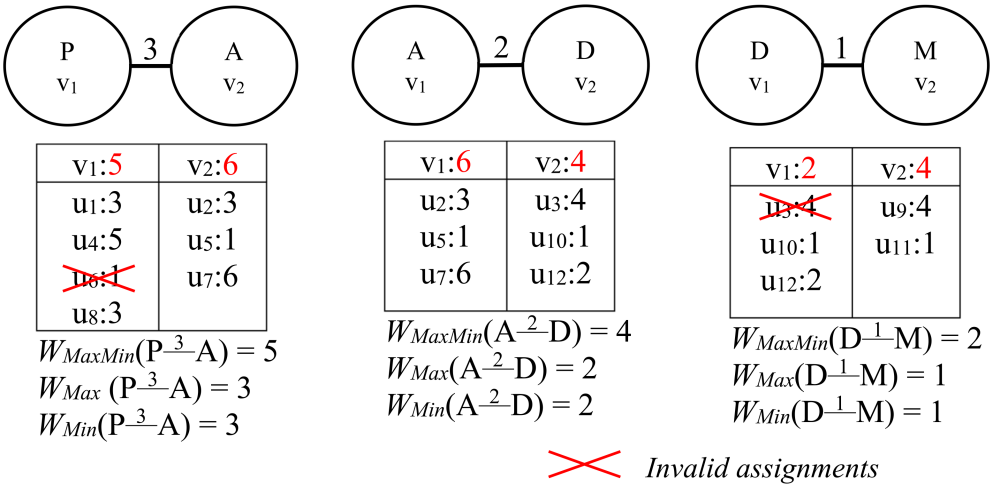


Figure 4. Valid and invalid assignments for the frequent edges.

Figure 1, we have three frequent edges in *fEdges*.

$$fEdges = \{P \overset{3}{—} A;\ A \overset{2}{—} D;\ D \overset{1}{—} M\}$$

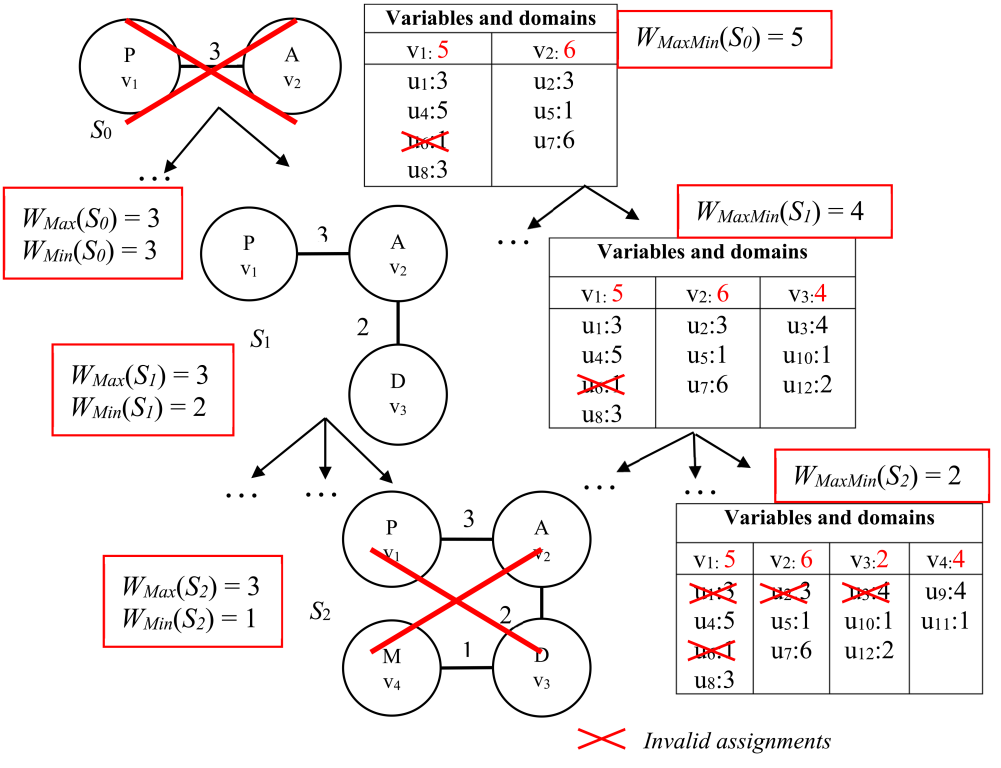


Figure 5. Generating phase for a frequent edge P $\overset{3}{—}$ A.

This list is also a list of 1-edge subgraphs with weights calculated as shown in Figure 4. Because the subgraphs in the *fEdges* list contain only one edge, the weight of this edge is also the weight of the entire subgraph.

As mentioned above, because the *MaxMin* method has been presented in detail in the WeGraMi (Le et al., 2020) and OWGraMi (Le et al., 2022) algorithms, in this paper our first contribution is to focus on presenting edge weights with the *Max* and *Min* methods. We will then use these *Max* and *Min* methods in our second contribution.

After getting the *fEdges* list, in the generating phase the WEGM algorithm will combine each edge *e* in the *fEdges* list to form candidate subgraphs *S*; In the weight calculation phase: each candidate subgraph *S* will be given support and weight. If *S* satisfies both frequency and weighting thresholds (set by the user), then *S* is a weighted subgraph. The process of generating and calculating will be performed recursively for subgraphs of *S* in the search tree (Figure 5).

Algorithm: WEGM

Input: A large weighted graph *G*, a frequency threshold *f* and a weighting threshold *w*
Output: All weighted subgraphs *S*
1 *resultList* ← Ø
2 Let *fEdges* list consists of all frequent edges *e* in *G*
3 **foreach** *e* ∈ *fEdges* **do**
4 if Weight(*e*, *w*) = 0 then //*Max or Min method by user*
5 remove *e*
6 **foreach** *e* ∈ *fEdges* **do**
7 *resultList* ← *resultList* ∪ Subgraph_Extend(*e*, *G*, *f*, *w*, *fEdges*)
8 remove *e*
9 **return** *resultList*

Algorithm: Subgraph_Extend

Input: A subgraph *S*, a large graph *G*, the frequency threshold *f*, the weighting threshold *w* and the set of frequent edges *fEdges* of *G*
Output: All weighted subgraphs of *G* that extend from *S*
1 result ← S
2 candidates ← Ø
3 **foreach** edge *e* ∈ *fEdges* and node *u* ∈ *S* **do**
4 **if** *e* can extend *u* **then**
5 Let *ex* be the extension of *S* with *e* by *u*
6 **if** *ex* is not already generated **then**
7 candidates ← candidates ∪ *ex*
8 **foreach** c ∈ candidates **do**
9 if $s_G(c) \geq f$ and **Weight**(*c*, *G*, *w*) ≥ 0 then
10 result ← result ∪ Subgraph_Extend (*c*, *G*, *f*, *w*, *fEdges*)
11 **return** result

In the second contribution, we propose early pruning of the edges *e* in the *fEdges* list that do not satisfy the weight threshold according to the *Max* or *Min* method. This has practical purposes for graph mining or social networks. For example: we need to find research groups in which members must have a large enough number of mutual articles (*Max* method), or find all product groups with few buyers in a year (*Min* method) to change the company's sales campaign.

In the WEGM algorithm, the Subgraph_Extend(*e*, *G*, *f*, *w*, *fEdges*) function (Line 7) is used to extend the weighted subgraphs *S* by adding each edge *e* in the *fEdges* list to generate new candidates, and this Subgraph_Extend() function similar to the OWGraMi algorithm. Line 3 to

line 7, the Subgraph_Extend() will generate all possible candidates by adding edges *e* in *fEdges* list to subgraph *S* by vertex *u*. Line 8 to Line 10, this function considers the generated candidates that satisfy the frequency threshold *f*, and the weighting threshold *w*, if generated candidate *c* can satisfy both *f* and *w*, the subgraph will be expanded recursively.

Our contribution is pruning the *fEdges* list (lines 3–5) before the generation process (lines 6–8). In *fEdges* list, we use the Weight() function to calculate the weight for 1-edge subgraphs *S* (corresponding to edges *e* in the *fEdges* list). Besides, we use two different functions to calculate weights for all subgraphs *S* (based on the user's purpose). The first function calculates weights according to the *Min* method and the second one is for the *Max* method.

Algorithm for Min method: Weight

Input: A subgraph *S* and a weighting threshold *w*
Output: The weight of *S* if this weight satisfies *w*, 0 otherwise
1 $W_S \leftarrow +\infty$ *//the weight of subgraph S*
2 **foreach** $e \in E_S$ **do**
3 $W_E \leftarrow$ the weight of e
4 **if** $W_E >= w$ **then** *//the weight of edge e does not satisfy w*
5 **return** 0 *//this means the weight of that subgraph can't satisfy w*
6 **else if** $W_S > W_E$ **then**
7 $W_S \leftarrow W_E$
8 **return** W_S

Algorithm for Max method: Weight

Input: A subgraph *S* and a weighting threshold *w*
Output: The weight of *S* if this weight satisfies *w*, 0 otherwise
1 $W_S \leftarrow 0$ *//the weight of subgraph S*
2 **foreach** $e \in E_S$ **do**
3 $W_E \leftarrow$ the weight of e
4 **if** $W_E < w$ **then** *//the weight of edge e does not satisfy w*
5 **return** 0 *//this means the weight of that subgraph can't satisfy w*
6 **else if** $W_S < W_E$ **then**
7 $W_S \leftarrow W_E$
8 **return** W_S

In line 2 of the two weighting methods mentioned above, the loop will traverse all edges *e* in the E_S edge list of the considered subgraph *S*. If the weight of a certain edge *e* does not satisfy the weight threshold *w* (line 4), then the entire subgraph does not satisfy the weight threshold *w* (line 5). If all edges *e* satisfy the weight threshold, the function will get the *Max* (or *Min*) value of all edges *e* in *S* (lines 6 and 7).

Because we use the *Max* (or *Min*) operator, if only one edge *e* in *S* does not satisfy the threshold, then the weight of *S* also does not satisfy the threshold, the programme will prune that edge in the *fEdges* list or prune that subgraph *S* on the search tree without affecting the final results.

When pruning edges early that do not satisfy the weight threshold in *fEdges*, WEGM will reduce the number of candidate subgraphs, thereby reducing the time to calculate support (NP-hard complexity Nguyen et al., 2022bb) for each candidate subgraph and the time to calculate the weight for each frequent subgraph.

For example, let the weight threshold $w = 2$, then the *fEdges* list according to *Max* and *Min* method will be:

$$fEdges = \{P \overset{3}{—} A;\ A \overset{2}{—} D;\ D \overset{1}{—} M\}$$
$$fEdges_{Max} = \{P \overset{3}{—} A;\ D \overset{2}{—} M\}$$
$$fEdges_{Min} = \{A \overset{2}{—} D;\ D \overset{1}{—} M\}$$

In Figure 5, this is a part of the search tree for the OWGraMi algorithm with three edges in the *fEdges* list. If the *Max* method is used, S_2 will not be generated because the edge D ^{1} M has been pruned early. If the *Min* method is used, sugraph S_0 will not be generated because the edge P ^{3} A has been pruned early (and of course S_1 and S_2 will not be generated either). Because we reduce the number of generated candidates during the mining process, our new algorithm significantly reduces running time and memory consumption.

5. Experiments

All our experiments are performed with Java SE Development Kit 8, Windows 10, Intel Core i5 CPU, four physical cores 3.2 GHz, eight threads, and 4GB RAM. We experiment on two data sets (directed and undirected graphs) and compare performance based on three criteria: search space, running time and memory consumption. We use the edge weighting threshold $w_v = 1$ for both algorithms to focus on the advantages of pruning candidate subgraphs with the edge weighting threshold w_e. In this section, we use three graph datasets (both directed and undirected graphs) as following:

(1) Facebook (Le et al., 2022; Nguyen et al., 2020): this is an undirected graph with 4,389 vertices and 88,235 edges, downloaded from http://snap.stanford.edu/data/. With 20 distinct vertex labels and 36 distinct edge labels (corresponding to edge weights in this WEGM algorithm), their rates are as in (Nguyen et al., 2020). Since the average value of edge weights in this dataset is 4.25, we use a weight threshold $w = 4$.
(2) p2p-Gnutella09 (Nguyen et al., 2022ba): This is a directed dataset with 8,114 vertices and 26,013 edges, downloaded from http://snap.stanford.edu/data/. This dataset is a sequence of snapshots for the peer-to-peer file sharing network Gnutella. They were collected in August 2002, with a total of nine Gnutella network snapshots. The vertices represent hosts in this network topology, and the edges represent the connections between these hosts. We randomly add 25 distinct vertex labels with 40 distinct edge labels, with the same ratings as in (Nguyen et al., 2022ba). Since the average value of edge weights in this dataset is 7.5393, we use $w = 8$ as the weighting threshold.
(3) Citeseer (Elseidy et al., 2014; Le et al., 2022): this is a directed graph with 3,312 vertices and 4,732 edges. With six vertex labels and edge labels from 0 to 100 (edge labels are the similarity between the corresponding pair of publications) and only take the

Table 1. The features datasets.

Dataset Names	Vertices	Node labels	Edges	Edge labels	Type
Facebook	4,389	20	88,235	1–36	Undirected
p2p-Gnutella09	8,114	25	26,013	1–40	Directed
Citeseer	3,312	6	4,732	0–100	Directed

integer value of the edge weight in this WEGM algorithm. Since the average value of edge weights in this dataset is 77.9081, we use $w = 78$ as the weighting threshold.

We use three datasets in Table 1, which are the same as in an earlier (Le et al., 2022), to ensure reliability when comparing the two algorithms.

First, we compare the effectiveness of the early pruning strategy for the search space based on the number of candidate subgraphs recorded during the experiments. The new WEGM algorithm demonstrates superiority in the search space on all datasets (directed and undirected graphs).

For the Facebook undirected graph in Figure 6(a), at a threshold of $f = 125$, using the *Min* operator to calculate weights and prune the search space can reduce the number of candidates by 53.3% (11,473–4,899), and even the *Max* operator only generated 212 candidates (corresponding to 1.85%). OWGraMi can only execute at a threshold of $f = 125$, and if the threshold is reduced OWGraMi will crash because of the memory constraint. WEGM with the *Min* operator can execute at $f = 115$ and crashes at the threshold $f = 110$, and the operator *Max* is the best in terms of the search space.

On the p2p-Gnutella09 dataset in Figure 6(b), the WEGM algorithm using the *Min* operator is the best. At a threshold of $f = 40$, the original OWGraMi needs 38,288 candidates, WEGM with the *Min* operator only needs 8,125 candidates (21.22%) and the *Max* operator only needs 4,153 candidates (10.85%). Besides, OWGraMi can only execute at $f = 40$, while our new algorithm can run at $f = 20$ with both *Max* and *Min* operators.

With the Citeseer directed graph in Figure 6(c), the WEGM algorithm with the *Min* operator is the best. At a threshold of $f = 13$, the OWGraMi algorithm generates 17,730 candidates, WEGM with the *Max* operator generates 7,591 candidates (42.81%) and the *Min* operator only generates 1,285 candidates (7.25%). Furthermore, OWGraMi can only execute at a threshold of $f = 13$, while the WEGM algorithm can run at a threshold lower than $f = 11$ with both *Max* and *Min* operators.

Because the WEGM algorithm can effectively prune the search space, the running time is also significantly reduced for all datasets.

On the Facebook dataset in Figure 7(a), at a threshold of $f = 125$ WEGM with the *Min* operator can reduce the running time by 55.51%, and the *Max* operator only needs 4.77% compared to the OWGraMi algorithm. In particular, if the threshold f is less than

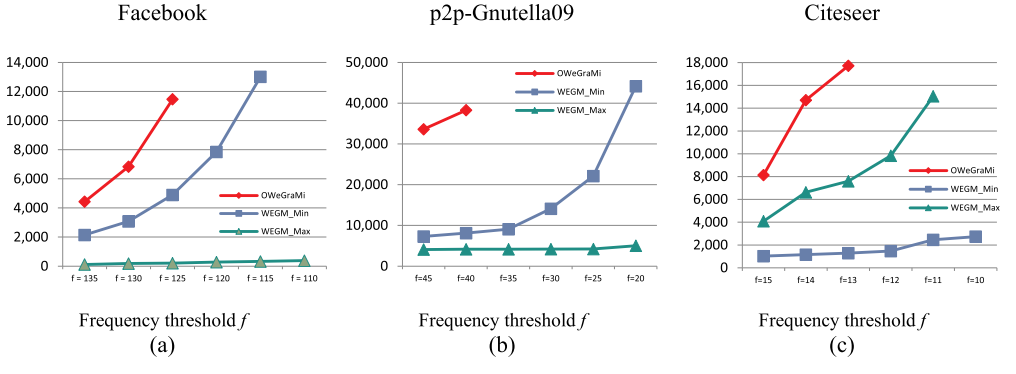


Figure 6. The number of candidate subgraphs.

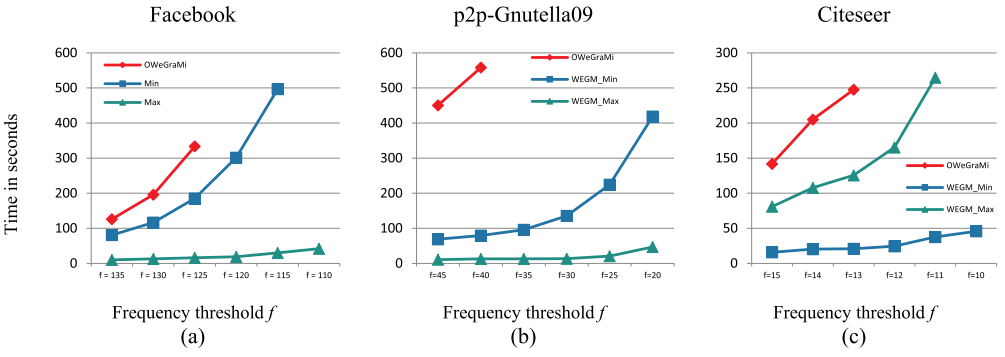


Figure 7. Running time for the two algorithms.

125, OWGraMi cannot perform due to the large amount of memory needed, while WEGM with both operators can run with lower thresholds.

With the p2p-Gnutella09 dataset in Figure 7(b), at a threshold of $f = 40$ our WEGM algorithm with the *Min* operator can reduce the running time by 14.21%, and the *Max* operator only needs 2.28% compared to the original OWGraMi algorithm. Furthermore, the two versions of the WEGM algorithm can both perform at lower thresholds of OWeGraMi in less time.

With the Citeseer dataset in Figure 7(c), our WEGM algorithm outperforms OWGraMi at all frequency thresholds. At a threshold of $f = 13$, WEGM with the *Max* operator only needs 50.67% of the running time compared to the original algorithm and the *Min* operator only needs 8.42%. And like the Facebook dataset, WEGM can perform at lower popularity thresholds than OWGraMi with both *Max* and *Min* operators.

Finally, we compare the memory consumption of the two algorithms. The WEGM algorithm has an effective pruning strategy for the search space, so the memory consumption of WEGM is significantly reduced compared to the original OWGraMi algorithm.

When experimenting on the Facebook dataset in Figure 8(a), the WEGM algorithm with the *Min* operator can reduce memory consumption to 30.93% of that needed by the OWGraMi algorithm at a threshold of $f = 125$, and with the *Max* operator it only needs

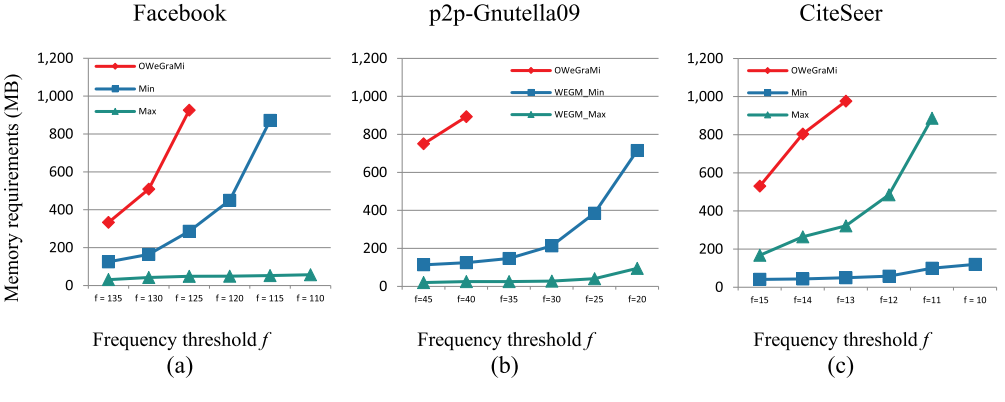


Figure 8. Memory consumption for all datasets.

5.28%. Moreover, while OWGraMi does not have enough memory to run at $f = 120$, the new algorithm can still run at lower thresholds.

For the p2p-Gnutella09 dataset in Figure 8(b), WEGM using the *Min* operator is the best algorithm. At a threshold of $f = 40$, the original OWGraMi needs 892.881 MB, WEGM with the *Min* operator only needs 13.96% of this compared to original algorithm and the *Max* operator only needs 2.85%. OWGraMi crashes at a threshold of $f = 35$, while the proposed algorithm WEGM can perform at $f = 20$.

With the Citeseer dataset in Figure 8(c), at a threshold of $f = 13$ the WEGM algorithm with the *Max* operator can reduce memory consumption to 33.02% of that needed by the OWGraMi algorithm and with the *Min* operator it only needs 5.11%. As with the datasets mentioned above, OWGraMi always crashes at higher thresholds than our new algorithm.

6. Conclusion

In this paper, our first contribution is to propose a new method to calculate weights for mined subgraphs based on their edges, such that the new algorithm can calculate the weights of both vertices and edges of all frequent subgraphs. Our second contribution is early pruning of unweighted candidates based on the proposed edge weights strategy. Our new algorithm, WEGM, has been proven theoretically and experimentally to have superior mining performance compared to the OWGraMi algorithm on all three evaluation criteria: search space, running time and memory consumption.

Because the current complexity of FSM problem is NP-hard, this is a big challenge for researchers, in the future, we will apply new pruning methods to reduce the complexity of graph mining algorithms, moreover we also aim to use parallel computing and cloud computing solutions to further improve mining performance based on the powerful capabilities of High-Performance Computing (HPC).

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

This work was supported by Kiengiang University (KGU) – Kiengiang Province, under [grant number 01/HĐKHCNCTGV-ĐHKG].

ORCID

Bay Vo http://orcid.org/0000-0002-9246-4587

References

Bui, H., Vo, B., Nguyen, H., Nguyen-Hoang, T.-A., & Hong, T.-P. (2018). A weighted N-list-based method for mining frequent weighted itemsets. *Expert Systems with Applications*, *96*, 388–405. https://doi.org/10.1016/j.eswa.2017.10.039

Chen, Z., He, C., Chen, G., Gan, W., & Fournier-Viger, P. (2024). HUSM: High utility subgraph mining in single graph databases. *Information Sciences*, *675*, 120743. https://doi.org/10.1016/j.ins.2024.120743

Corso, G., Stark, H., Jegelka, S., Jaakkola, T., & Barzilay, R. (2024). Graph neural networks. *Nature Reviews Methods Primers*, *4*(1), 17. https://doi.org/10.1038/s43586-024-00294-7

Dai, G., Zhu, Z., Tianyu, F., Wei, C., Wang, B., Li, X., Xie, Y., Yang, H., & Wang, Y., (2022). DIMMining: pruning-efficient and parallel graph mining on near-memory-computing. in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, in ISCA '22. New York, NY, USA: Association for Computing Machinery, pp. 130–145. https://doi.org/10.1145/3470496.3527388

Elseidy, M., Abdelhamid, E., Skiadopoulos, S., & Kalnis, P. (2014). Grami: Frequent subgraph and pattern mining in a single large graph. *Proceedings of the VLDB Endowment*, *7*(7), 517–528. https://doi.org/10.14778/2732286.2732289

Fan, W., Wang, X., Wu, Y., & Xu, J. (2015). Association rules with graph patterns. *Proceedings of the VLDB Endowment*, *8*(12), 1502–1513. https://doi.org/10.14778/2824032.2824048

Fournier-Viger, P., Cheng, C., Lin, J. C.-W., Yun, U., & Kiran, R. U. (2019a). Tkg: Efficient mining of top-k frequent subgraphs. In *International Conference on Big Data Analytics*, Springer, pp. 209–226.

Fournier-Viger, P., Zhang, Y., Lin, J. C.-W., Fujita, H., & Koh, Y. S. (2019b). Mining local and peak high utility itemsets. *Information Sciences*, *481*, 344–367. https://doi.org/10.1016/j.ins.2018.12.070

Franciscus, N., Ren, X., & Stantic, B. (2019). Dependency graph for short text extraction and summarization. *J. Inf. Telecommun.*, *3*(4), 413–429.

Guzmán-Ponce, A., Marcial-Romero, J. R., Valdovinos-Rosas, R. M., & Sánchez-Garreta, J. S. (2020). Weighted complete graphs for condensing data. *Electronic Notes in Theoretical Computer Science*, *354*, 45–60. https://doi.org/10.1016/j.entcs.2020.10.005

He, C., Chen, X., Chen, G., Gan, W., & Fournier-Viger, P. (2024). Mining credible attribute rules in dynamic attributed graphs. *Expert Systems with Applications*, *246*, 123012. https://doi.org/10.1016/j.eswa.2023.123012

Huynh, B., Nguyen, L. B. Q., Nguyen, D. H. M., Nguyen, N. T., Nguyen, H.-S., Pham, T., Pham, T., Nguyen, L. T. T., Nguyen, T. D. D., & Vo, B. (2024). Mining association rules from a single large graph. *Cybernetics and Systems*, *55*(3), 693–707. https://doi.org/10.1080/01969722.2022.2162740

Lam B.Q., Nguyen. (2021b). CCGraMi: An Effective Method for Mining Frequent Subgraphs in a Single Large Graph. *MENDEL*, 27(2), 90–99. https://doi.org/10.13164/mendel.2021.2.090

Le, N.-T., Vo, B., Nguyen, L. B. Q., Fujita, H., & Le, B. (2020). Mining weighted subgraphs in a single large graph. *Information Sciences*, *514*, 149–165. https://doi.org/10.1016/j.ins.2019.12.010

Le, N.-T., Vo, B., Nguyen, L. B. Q., & Le, B. (2022). OWGraMi: Efficient method for mining weighted subgraphs in a single graph. *Expert Systems with Applications*, *204*, 117625.

Le, N.-T., Vo, B., Yun, U., & Le, B. (2023). A novel approach to discover frequent weighted subgraphs using the average measure. *Applied Intelligence*, *53*(16), 19491–19504. https://doi.org/10.1007/s10489-023-04501-y

Li, Y., Zhou, Z., Li, Q., Li, T., Julian, I. N., Guo, H., & Chen, J. (2022). Depression classification using frequent subgraph mining based on pattern growth of frequent edge in functional magnetic resonance imaging uncertain network. *Frontiers in Neuroscience*, *16*, 889105. https://doi.org/10.3389/fnins.2022.889105

Liu, N., Li, D., Zhang, Y., & Li, X. (2020). Large-scale graph processing systems: a survey. *Frontiers of Information Technology & Electronic Engineering*, *21*(3), 384–404. https://doi.org/10.1631/FITEE.1900127

Nguyen, L. B. Q. (2022). Efficient methods for mining subgraphs in a single large graph. PhD Thesis, VSB – Technical University of Ostrava.

Nguyen, L. B. Q., et al. (2022ba). An efficient and scalable approach for mining subgraphs in a single large graph. *Appl. Intell.*, *52*(15), 17881–17895.

Nguyen, L. B. Q., Le, N.-T., Nguyen, H. S., Pham, T., & Vo, B. (2022aa). Frequent closed subgraph mining: A multi-thread approach. in *Intelligent Information and Database Systems: 14th Asian Conference, ACIIDS 2022, Ho Chi Minh City, Vietnam, November 28–30, 2022, Proceedings, Part I*, Springer, pp. 64–77.

Nguyen, L. B. Q., Nguyen, L. T. T., Zelinka, I., Snasel, V., Nguyen, H. S., & Vo, B. (2021a). A method for closed frequent subgraph mining in a single large graph. *IEEE Access*, *9*, 165719–165733.

Nguyen, L. B. Q., Vo, B., Le, N.-T., Snasel, V., & Zelinka, I. (2020). Fast and scalable algorithms for mining subgraphs in a single large graph. *Engineering Applications of Artificial Intelligence*, *90*, 103539. https://doi.org/10.1016/j.engappai.2020.103539

Nguyen, L. T. T., Vu, V. V., Lam, M. T. H., Duong, T. T. M., Manh, L. T., Nguyen, T. T. T., Vo, B., & Fujita, H. (2019). An efficient method for mining high utility closed itemsets. *Information Sciences*, *495*, 78–99. https://doi.org/10.1016/j.ins.2019.05.006

Nguyen, L. B. Q., Zelinka, I., & Diep, Q. B. (2021b). CCGraMi: An effective method for mining frequent subgraphs in a single large graph. *MENDEL*, *27*(2), 90–99. https://doi.org/10.13164/mendel.2021.2.090

Nguyen, L. B. Q., Zelinka, I., Snasel, V., Nguyen, L. T. T., & Vo, B. (2022bb). Subgraph mining in a large graph: A review. *WIREs Data Mining and Knowledge Discovery*, *12*(4), e1454. https://doi.org/10.1002/widm.1454

Ning, X., Wang, Y., Yu, K., Miao, J., Cao, F., & Liang, J. (2025). Summary Graph Induced Invariant Learning for Generalizable Graph Learning. *IEEE Transactions on Knowledge and Data Engineering* , *37*(6), 3739–3752. https://doi.org/10.1109/TKDE.2025.3547226

Park, C., Park, H.-M., & Kang, U. (2020). FlexGraph: Flexible partitioning and storage for scalable graph mining. *PLoS One*, *15*(1), e0227032.

Preti, G., De Francisci Morales, G., & Riondato, M. (2023). MaNIACS: Approximate mining of frequent subgraph patterns through sampling. *ACM Transactions on Intelligent Systems and Technology*, *14*(3), 1–29. https://doi.org/10.1145/3587254

Preti, G., Lissandrini, M., Mottin, D., & Velegrakis, Y. (2021). Mining patterns in graphs with multiple weights. *Distributed and Parallel Databases*, *39*(2), 281–319. https://doi.org/10.1007/s10619-019-07259-w

Ren, T., Zhang, H., Wang, Y., Ju, W., Liu, C., Meng, F., Yi, S., & Luo, X. (2025). MHGC: Multi-scale hard sample mining for contrastive deep graph clustering. *Information Processing & Management*, *62*(4), 104084. doi:https://doi.org/10.1016j.ipm.2025.104084

Sahu, S., Chawla, M., Khare, N., & Singh, B. (2023). Mining approximate frequent subgraph with sampling techniques. *Materials Today: Proceedings*, *81*, 395–402. doi:https://doi.org/10.1016j.matpr.2021.03.425

Sasaki, Y., & Karras, P. (2024). Mining path association rules in large property graphs. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, in CIKM '24. New York, NY, USA: Association for Computing Machinery, pp. 1994–2003. https://doi.org/10.1145/3627673.3679525.

Svitáková, L., Valenta, M., & Pokorný, J. (2021). Enhanced adaptive partitioning in a distributed graph database. *J. Inf. Telecommun.*, *5*(1), 104–120.

Talukder, N., & Zaki, M. J. (2016). Parallel graph mining with dynamic load balancing. In 2016 *IEEE International Conference on Big Data (Big Data)*, IEEE, 2016, pp. 3352–3359.

Wang, X., Xu, Y., & Zhan, H. (2020). Extending association rules with graph patterns. *Expert Systems with Applications*, *141*, 112897. https://doi.org/10.1016/j.eswa.2019.112897

Wangmo, C., & Wiese, L. (2023). SuMExplorer: Summarisation-based frequent subgraph mining for visual exploratory subgraph searching. *LWDA*, *3630*, pp. 110–120.

Zhong, B. (2024). Fine-grained sentiment analysis using multidimensional feature fusion and GCN. *Journal of Information and Telecommunication*, *9*(1), 1–22.